

fragment. It allows other objects to observe the life cycle.

This component is used to add life cycle awareness. It will be helpful for library makers to clear up their resources, depending upon the life cycle.

Let us now see how the Lifecycle component helps. Here is an example of how we write networking code:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Network call
    api.get().onResponse { data ->
        textView.text = data
    }
}
```

 **Note:** The code snippet given above is in Kotlin, as it's now the official language for building Android apps.

It's a simple networking code which sends a GET request to the Web and sets the response to the TextView. It looks simple, but sadly it's not, because when you rotate the device or close the app, the request will still be running in the background, and on getting a response, will crash because TextView doesn't exist anymore as the Activity is already destroyed.

Android developers are aware of this problem as the life cycle is controlled by the system. To solve this problem, we need to cancel the request when Activity is destroyed, as shown below:

```
override fun onCreate(savedInstanceState: Bundle?) {
    ///....
    // Network call
    request = api.get()
    request.onResponse { data ->
        textView.text = data
        request = null
    }
}

override fun onDestroy() {
    if (request != null) request.cancel()
}
```

However, as the complexity of the application increases, this becomes quite challenging to manage and is very easy to miss, not to mention the extra boilerplate code.

The Lifecycle component is designed to solve such problems by making components life cycle-aware (so that they are aware about the state of the Activity). Let's look at the same code snippet using the Lifecycle library:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
```

```
setContentView(R.layout.activity_main)
// Network call
api.get().onResponse(getLifecycle()) { data ->
    textView.text = data
}
}
```

 **Note:** This is an imaginary networking code showing how other libraries can observe the life cycle and automatically clean themselves up.

Anatomy of the Lifecycle component: There are two main parts to the Lifecycle component—Lifecycle owners and Lifecycle observers.

1. *Lifecycle owners* are the UI controllers, which have their own life cycle—for example, Activity and Fragment.
2. *Lifecycle observers* observe Lifecycle owners and are notified about life cycle changes. We use 'LifecycleObserver' to build life cycle-aware components.

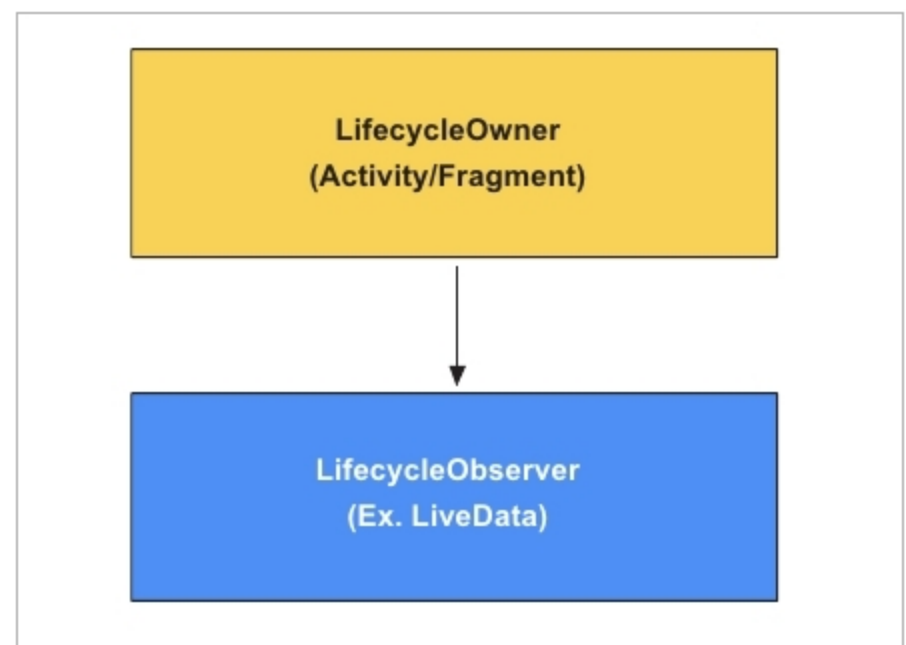


Figure 1: Lifecycle component

LiveData

LiveData is used to instantly notify changes in the data to the UI. The following list describes what LiveData is:

- An observable data holder
- It notifies the observers when data changes so that it can update the UI
- Respects Lifecycle, i.e., it is life cycle-aware
- It is pretty similar to RxJava

Overall flow of LiveData: The UI components observe changes in LiveData, which in turn observes Lifecycle owners, i.e., Activity or Fragment for Lifecycle.

Here is how LiveData works internally. It observes for updates or changes in the data and when the value is updated, it checks if the UI controller—i.e., Activity or Fragment—is in the STARTED or RESUMED state (which means the UI is shown to the user) to notify the observer to update the UI.